



Facultad de Ingeniería

Diseño de Aplicaciones 2

Obligatorio 1 – Descripción del diseño

“Entregado como requisito para la obtención del título de Ingeniero en Sistemas”

Repo: <https://github.com/IngSoft-DA2/271395-294598-221509.git>

Candelaria Blanco - 271395

Maximiliano Gimenez - 294598

Ana Betina Kadessian - 221509

Tutores:

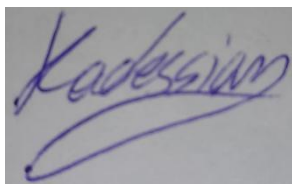
Nicolás Fierro, Maximo Healty

2025

DECLARACIÓN DE AUTORÍA

Nosotros, Candelaria Blanco, Maximiliano Gimenez y Ana Betina Kadessian, declaramos que el trabajo que se presenta en esa obra es de nuestra propia mano. Podemos asegurar que:

- La obra fue producida en su totalidad mientras realizábamos el obligatorio 1 de la materia Diseño de Aplicaciones 2.
- Cuando hemos consultado el trabajo publicado por otros, lo hemos atribuido con claridad
- Cuando hemos citado obras de otros, hemos indicado las fuentes. Con excepción de estas citas, la obra es enteramente nuestra
- En la obra, hemos acusado recibo de las ayudas recibidas
- Cuando la obra se basa en trabajo realizado conjuntamente con otros, hemos explicado claramente qué fue contribuido por otros, y qué fue contribuido por nosotros
- Ninguna parte de este trabajo ha sido publicada previamente a su entrega, excepto donde se han realizado las aclaraciones correspondientes.



Ana Betina Kadessian

8/5/2025



Candelaria Blanco

8/5/2025



Maximiliano Gimenez

8/5/2025

ABSTRACT

Este trabajo presenta el desarrollo de un Simulador de Objetos que permite modelar y ejecutar estructuras propias de la Programación Orientada a Objetos (POO). El sistema posibilita la creación de clases con herencia simple, definición de atributos y métodos, y ejecución de métodos respetando accesibilidad, herencia y polimorfismo. También permite simular patrones de diseño como Template Method y Strategy. La solución fue implementada como una API REST utilizando .NET 8 y Entity Framework Core, siguiendo buenas prácticas de diseño, Clean Code y TDD. Este proyecto fue desarrollado para la Universidad ORT Uruguay como parte de la materia Diseño de Aplicaciones 2, con el objetivo de aplicar conceptos avanzados de diseño y arquitectura de software en un entorno práctico y educativo.

Índice

Descripción general	4
Qué hace la solución	4
Bugs conocidos	4
Funcionalidades no implementadas	4
Diagrama general de paquetes	5
Breve descripción de responsabilidades de cada paquete	5
Diagrama de clases de cada paquete	6
Descripción de jerarquías de herencia utilizadas.....	11
Modelo de tablas de la estructura de la base de datos	12
Funcionalidades de mayor relevancia	12
Diagramas de interacción que muestran las clases involucradas	12
Justificación del diseño	14
Mecanismos de inyección de dependencias	15
Principios SOLID.....	15
Otros principios de diseño utilizados.....	17
Decisiones propias de diseño	17
Descripción del mecanismo de acceso a datos	18
Descripción del manejo de excepciones.....	19
Diagrama de componentes	20

Descripción general

Qué hace la solución

Como resultado, desarrollamos un sistema que emula el funcionamiento de un entorno de programación orientada a objetos, brindando la posibilidad de modelar clases, objetos, atributos, métodos y sus ejecuciones, respetando las principales reglas y restricciones propias de este paradigma. Entre ellas se incluyen aspectos como la herencia, los niveles de visibilidad y el comportamiento específico de distintos tipos de clases (abstractas, selladas, etc.).

El sistema permite construir jerarquías de clases utilizando herencia simple, definir atributos y métodos con sus tipos y parámetros correspondientes, y simular la ejecución de métodos.

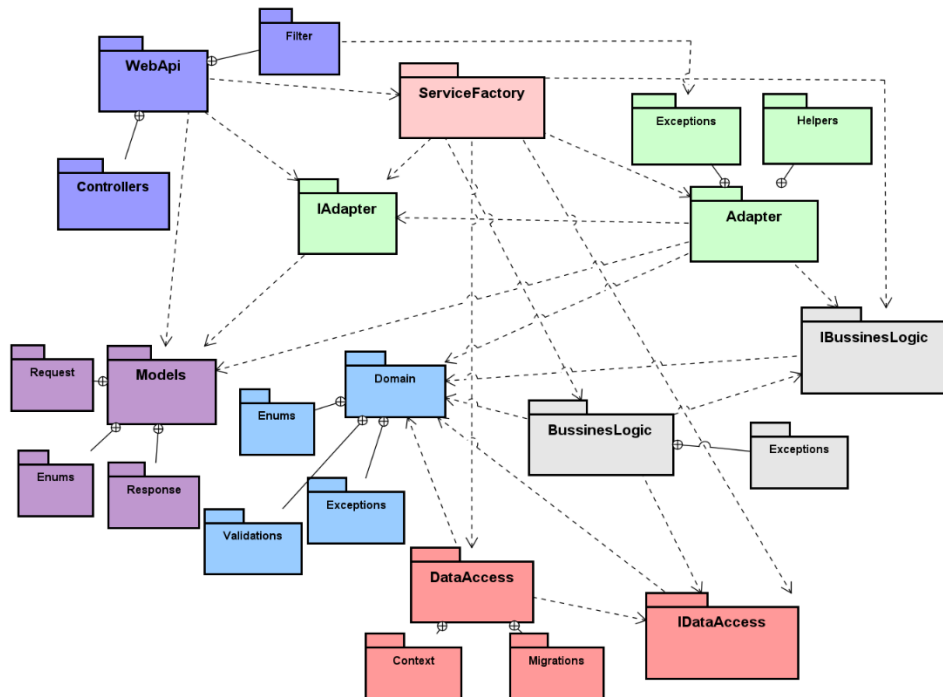
Bugs conocidos

Al realizar un update de una SimClass a través de dicho endpoint se pueden crear atributos y métodos con el mismo nombre en una misma clase.

Funcionalidades no implementadas

Al momento de documentar nos dimos cuenta de que la ejecución de métodos con los ejemplos dados ejecuta correctamente, pero falta una validación que es que, si el método que se quiere ejecutar no está en la clase de Referencia, y si en la del Objeto a crear se ejecuta igual, debería lanzar un error en ese caso, no se pudo implementar por falta de tiempo.

Diagrama general de paquetes



Breve descripción de responsabilidades de cada paquete

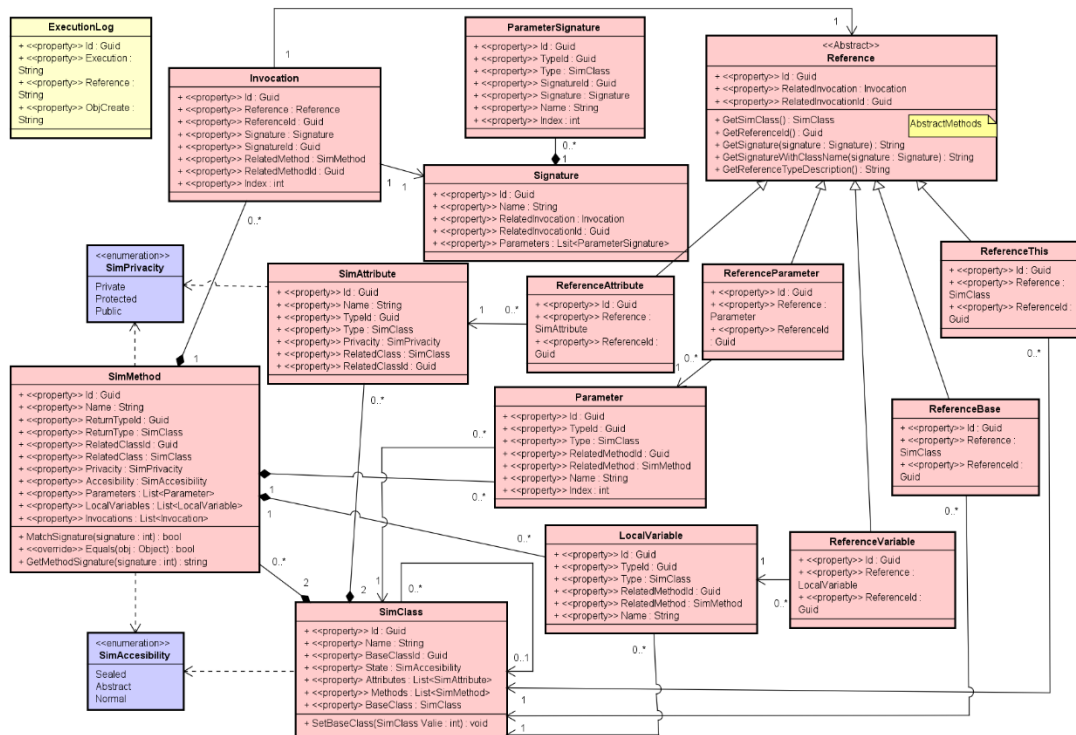
- **WebApi:** proporciona la capa de API web para la aplicación, define los controladores y la configuración para exponer la API REST. Actúa como la capa de presentación para clientes externos. Maneja solicitudes y (mapea) respuestas HTTP (JSON). Expone la funcionalidad del sistema mediante endpoints RESTfull accesibles vía HTTP.
- **IAdapter:** Define la interfaz (contrato) que expone los métodos utilizados por la WebAPI. Establece qué operaciones están disponibles, sin revelar detalles de implementación. Permite programar contra una abstracción, facilitando el desacoplamiento y la testabilidad.
- **Adapter:** Implementa la interfaz IAdapter. Actúa como puente entre la capa de presentación (WebAPI) y la lógica de negocio. Recibe servicios a través de la interfaz IBussinesLogic y se encarga de transformar los datos de Models que recibe de la WebApi a elementos de Domain y viceversa. Aplica el patrón Adapter.
- **Domain:** contiene la definición de las entidades principales del sistema.
- **Models:** contiene las clases de modelos para solicitudes (*Request*) y respuestas (*Response*) necesarios para la comunicación de la WebApi.

Encapsula únicamente los datos que se quieran mostrar a la capa de presentación.

- **IBusinessLogic:** define las interfaces para la lógica de negocio, actúa como un intermediario para que los proyectos cuyas funcionalidades dependen de la lógica de negocio no interactúen directamente con su implementación. Permite la inversión de dependencias, facilitando pruebas y mantenimiento.
- **BusinessLogic:** contiene la lógica principal del negocio, define como se comporte el sistema según los requerimientos. Es independiente de las tecnologías de presentación o persistencia, ya que como mencionamos anteriormente, solo depende de los contratos definidos en la interfaz. Se encarga de coordinar las operaciones complejas del sistema (flujo de datos entre entidades o servicios del dominio).
- **IDataAccess:** define los contratos para acceder a las fuentes de datos, es el intermediario entre la lógica de negocio y el acceso a los datos. Facilita el uso de inyección de dependencias para permitir que la lógica de negocio utilice acceso a datos sin conocer su implementación.
- **DataAccess:** implementa la lógica para acceder y manipular datos de la base de datos, depende de *IDataAccess* para desacoplar el acceso a datos del resto del sistema; demás capas no tienen acceso directo a la base de datos haciéndolo más seguro. Aplica el principio de inversión de dependencias. Contiene migraciones, *DbContext* y mapeos entidad-tabla.
- **ServiceFactory:** tiene como objetivo principal centralizar la configuración y el registro de servicios que serán utilizados. El método *AddServices* de la clase *SimulatorServiceFactory*, actúa como punto de entrada para registrar y configurar todos los servicios requeridos por la WebApi, tales como adaptadores, componentes de lógica de negocio y mecanismos de acceso a datos. Su fin es mantener la arquitectura desacoplada, desacoplamos la WebApi de *DataAccess*, *IDataAccess*, *BusinessLogic*, *IBusinessLogic* y *Adapter*. Sigue el patrón de inyección de dependencias facilitando la escalabilidad de los distintos componentes del sistema. Centraliza la configuración en un único lugar, mejorando la claridad del sistema al momento de inicializar el entorno de simulación.

Diagrama de clases de cada paquete

Domain:



Para definir la representación de una clase definimos una clase llamada **“SimClass”**, esta contiene un atributo *BaseClass* que hace alusión a un atributo de la misma clase SimClass esto para simular el mecanismo de herencia, además esta clase contiene los métodos que son otra clase **“SimMethod”** y también contiene sus atributos **“SimAttributes”**, dentro de los métodos hay más clases como una lista de **“LocalVariable”** representando variables locales de un método, de **“Parameter”** representando los parámetros de un método, y de **“Invocation”** representando las llamadas que realiza ese método a otros métodos.

Nos enfrentamos al reto de saber cómo resolver la simulación de ejecución de métodos por medio del polimorfismo, ya que podía haber override, entonces asociar las invocaciones a él Id de un método específico nos pareció una opción poco conveniente ya que este iba cambiando, dependiendo si se sobrescribía en clases superiores. Entonces decidimos que para saber a qué método se iba a llamar en una invocación se, detalle el nombre del método y sus parámetros en una nueva clase llamada **“Signature”** esa clase representa eso una firma de un método, podríamos haber utilizado la clase Parameter también en vez de crear **“ParameterSignature”**, pero queríamos diferenciarlo y que no se asocie a un id de un método específico, para dejar bien en claro el dominio del proyecto, además esta clase nos permite acercarnos un poco a la ejecución real de un método, ya que gracias a ParameterSignature un usuario puede poner el nombre de parámetros que quiera al realizar una invocación, lo que realmente distingue a un método es su nombre, el tipo de clase de sus parámetros y su orden.

En el caso de las invocaciones de métodos, observamos que necesitábamos representar desde dónde se realiza la llamada. Para esto, utilizamos una clase

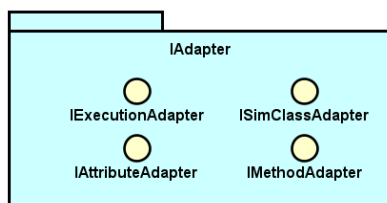
abstracta llamada **Reference**, que permite modelar distintas formas de referencia mediante el uso de polimorfismo. Sus subclases son:

- 1- **ReferenceThis**: representa llamadas del tipo `this.Metodo()`. Esta subclase contiene como atributo una instancia de `Reference`, que hace referencia a la `SimClass` desde la cual se ejecuta ese `this`.
- 2- **ReferenceBase**: al igual que en un lenguaje de programación real esta subclase hace alusión a el tipo de llamadas `base.Metodo()`. Esta subclase contiene como atributo una `SimClass` tambien desde la cual se ejecuta ese `base`, la diferencia con la anterior es que al obtener el tipo a través del método sobrescrito “`GetClass()`”, nos dará la clase base de el atributo del cual se llama.
- 3- **ReferenceAttribute**: representa las llamadas de esa invocación desde un atributo, como atributo `Reference` contiene el `SimAttribute` relacionado, y al llamarse la sobreescritura del método “`GetClass()`” nos da como resultado la `SimClass` del tipo del atributo, ya que así podemos relacionar los métodos del tipo del atributo con un atributo.
- 4- **ReferenceParameter**: representa las llamadas desde un parámetro de un metodo y como atributo `Reference` contiene el `Parameter` relacionado, al igual que el anterior al llamar a “`GetClass()`” nos devuelve la `SimClass` del tipo del parametro.
- 5- **ReferenceVariable**: representa las llamadas desde una variable local de un método, como atributo `Reference` contiene la `LocalVariable` relacionada, al llamar a “`GetClass()`” nos devuelve la `SimClass` del tipo de la variable.

De esta forma en la clase *Invocation*, con una clase *Reference* asociada y una clase *Signature* tendríamos información suficiente para ir a buscar el método al que hace alusión. Para esto, implementamos una búsqueda recursiva, que sigue la cadena de herencias hasta encontrar el método adecuado en la clase correspondiente, considerando posibles sobrescrituras (override). Elegimos este enfoque porque evita tener que mantener actualizadas listas de invocaciones manualmente cada vez que cambia la jerarquía de clases o se redefine un método.

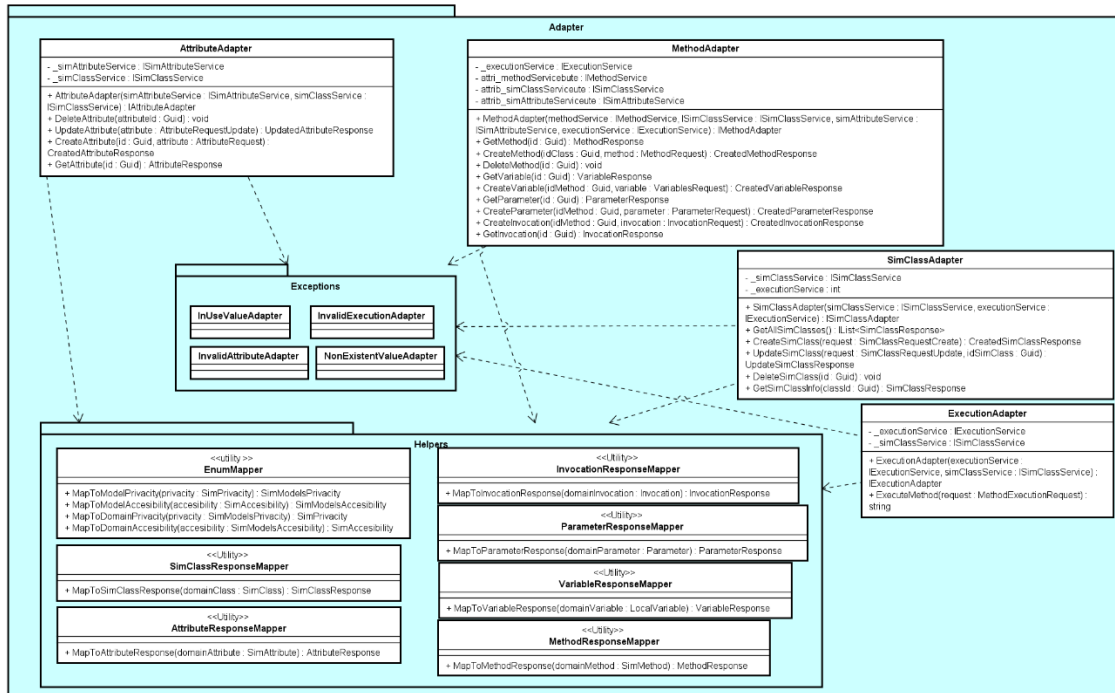
Por último, contamos con una clase llamada *ExecutionLog*, que es una clase independiente que guarda los registros de ejecuciones realizadas por el sistema.

IAdapter:



Paquete de interfaces IAdapter.

Adapter:

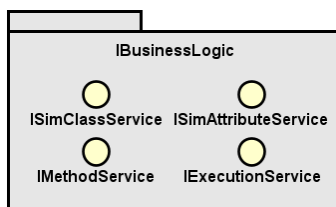


En nuestro diseño UML, implementamos el patrón Adapter para desacoplar la lógica de negocio (BusinessLogic) de los models específicos que usa la WebAPI.

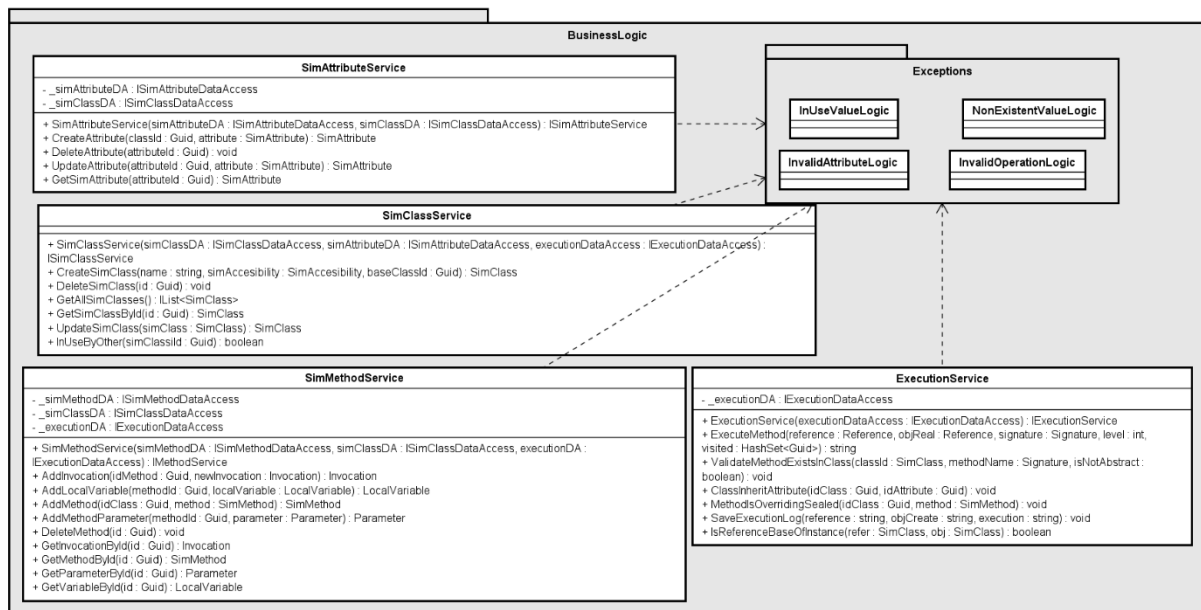
Creamos interfaces como ISimClassAdapter (IAdapter) que son utilizadas por la lógica de negocio.

Los adapters concretos (SimClassAdapter) transforman los objetos del dominio interno en los models que usa la WebAPI (Adaptado), permitiendo que la lógica de negocio opere sin conocer los detalles de los DTOs de la API.

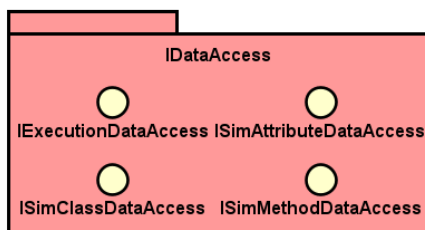
IBusinessLogic:



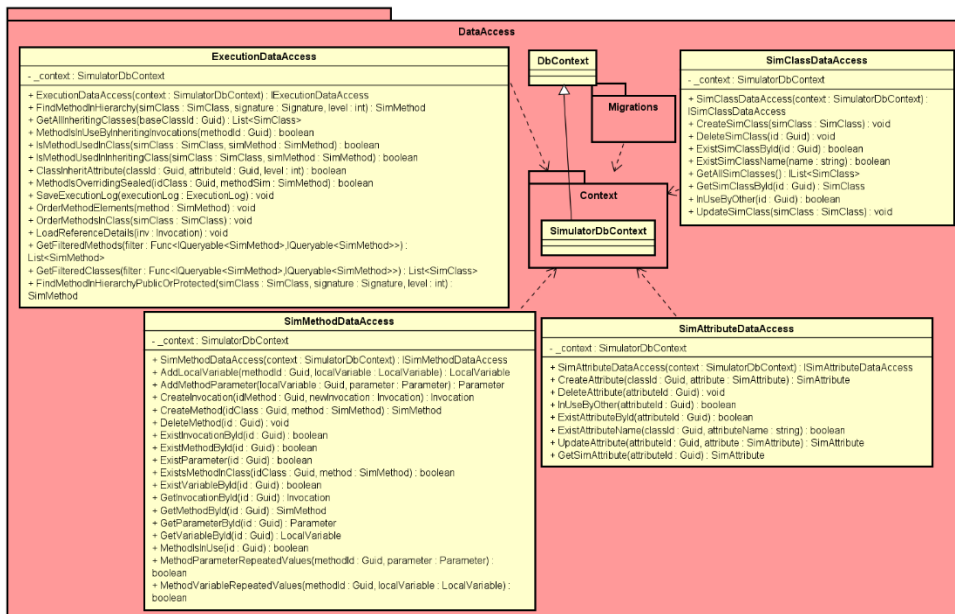
BusinessLogic:



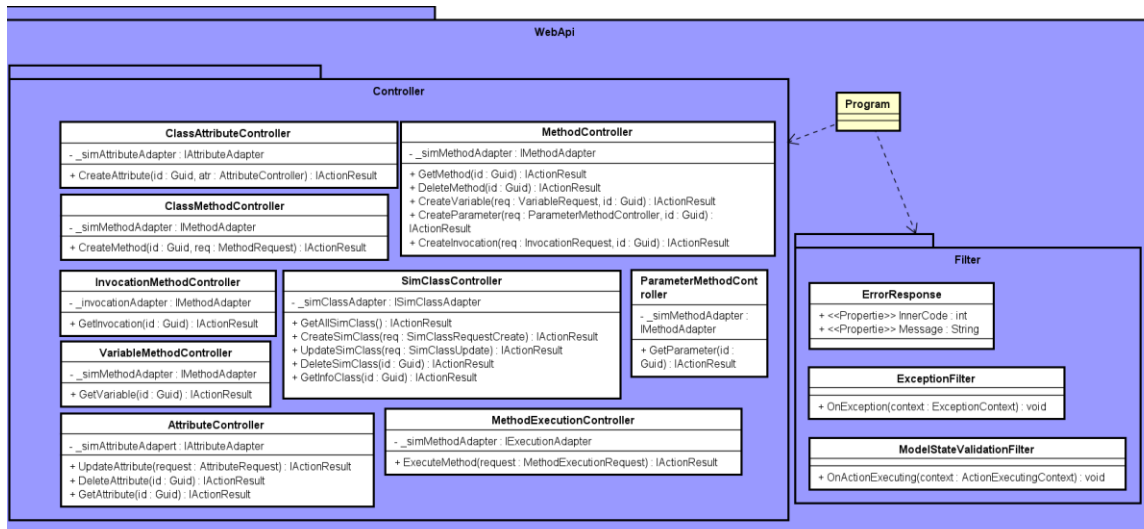
IDataAccess:



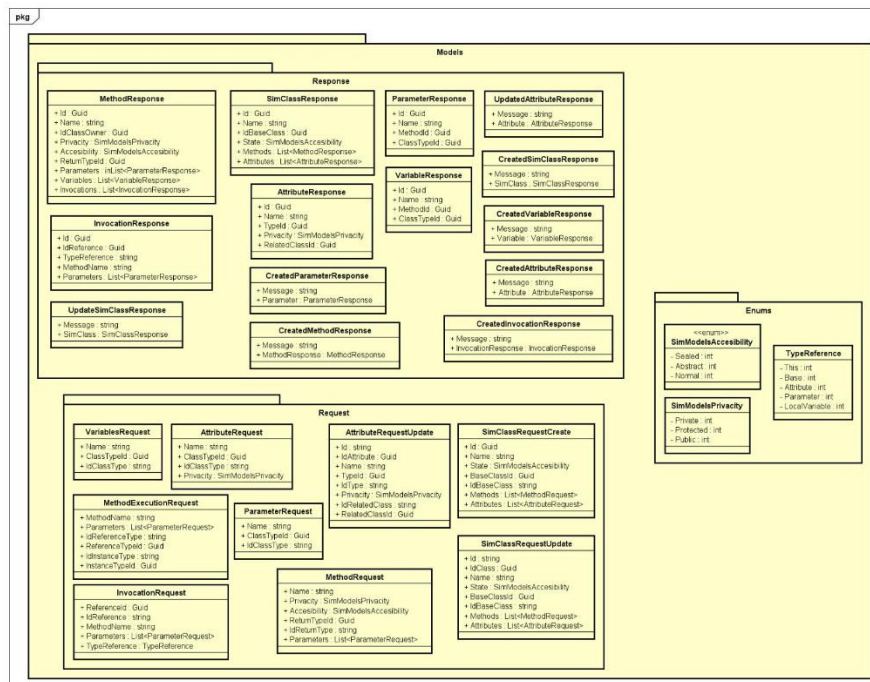
DataAcces:



WebApi



Models:



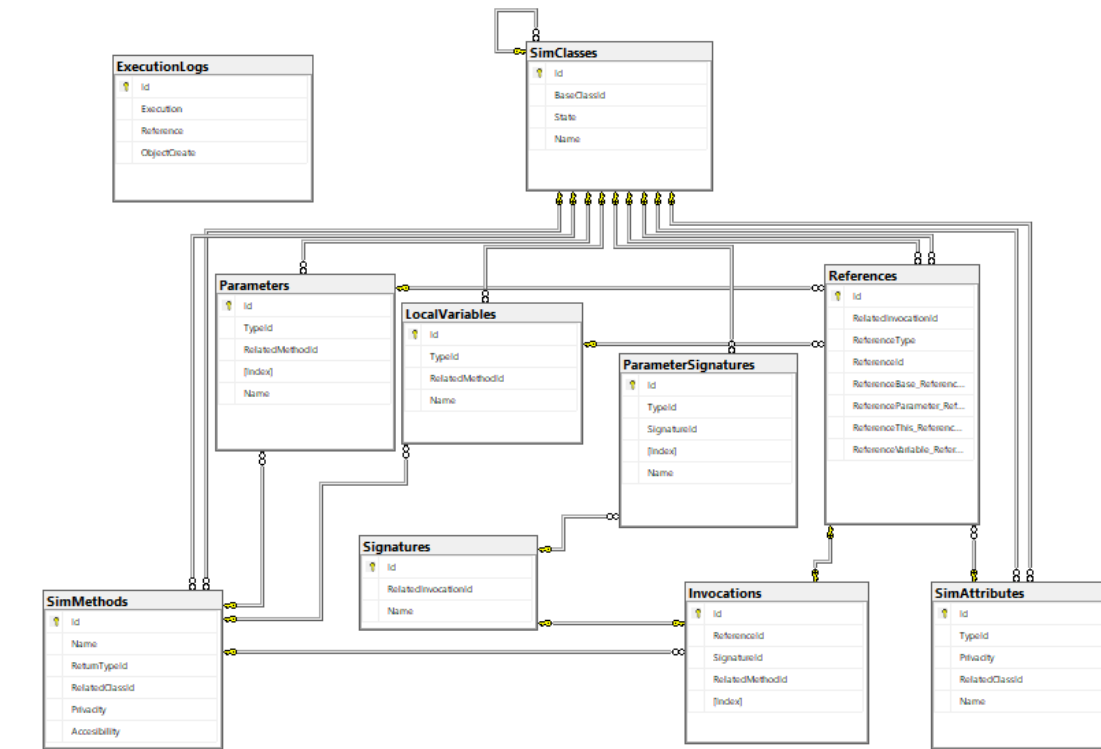
Descripción de jerarquías de herencia utilizadas

Se utilizó herencia en el caso de la clase “Reference”, implementando una clase abstracta y cinco subclases concretas: ReferenceThis, ReferenceBase, ReferenceVariable, ReferenceParameter y ReferenceAttribute. Esta jerarquía permitió aprovechar el polimorfismo para definir comportamientos específicos en cada subclase, facilitando la ejecución diferenciada según el tipo de referencia.

El uso de herencia redujo la duplicación de código y contribuyó a un diseño más claro y mantenible, especialmente dada la complejidad de simular la ejecución de métodos polimórficos. Además, este enfoque respeta el principio de Open/Closed del modelo SOLID: el sistema está abierto a la extensión, pero cerrado a la modificación. Esto

significa que, si en el futuro se requiere soportar un nuevo tipo de referencia (por ejemplo, clases estáticas), se podría agregar una nueva subclase sin alterar el comportamiento existente.

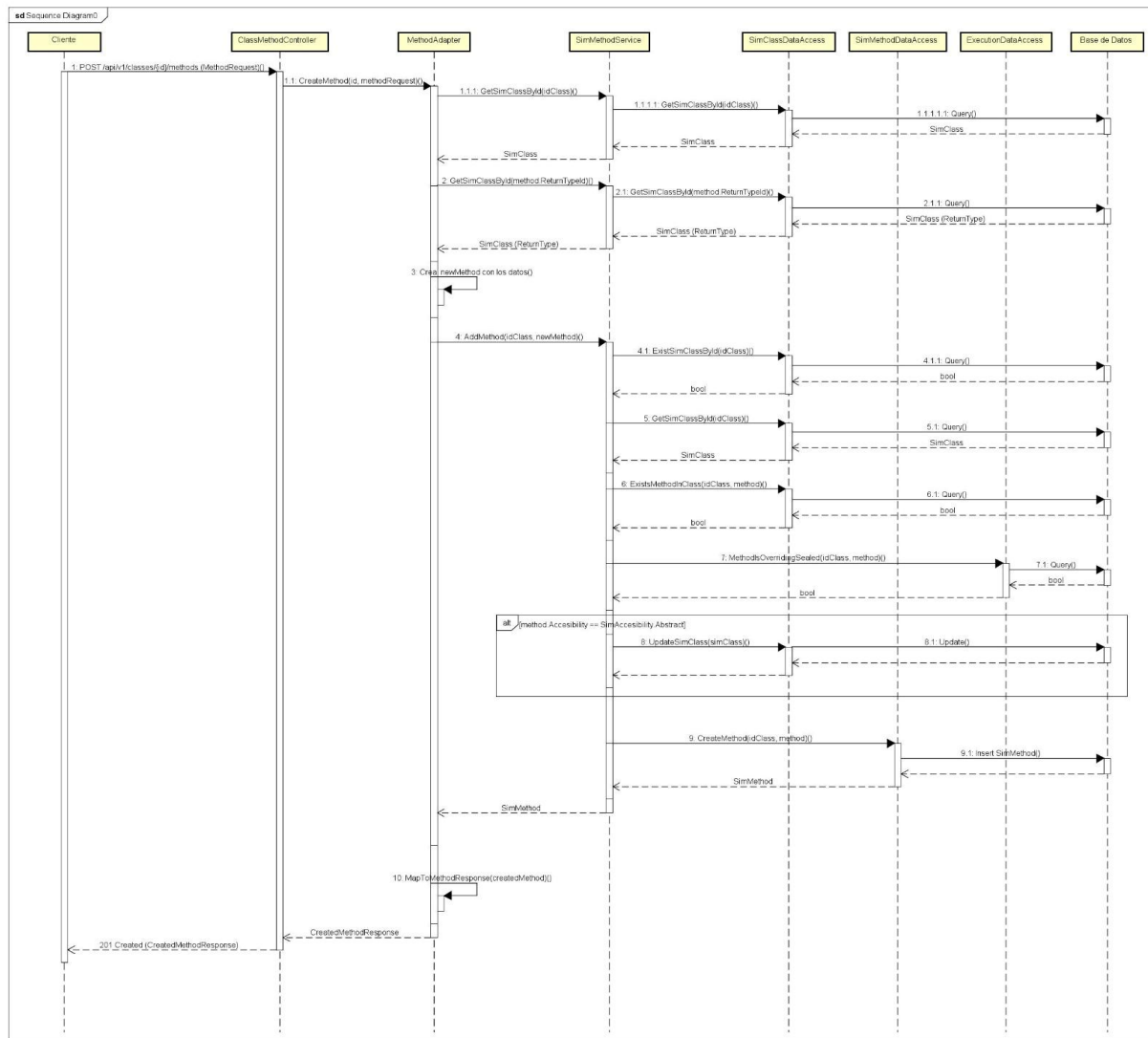
Modelo de tablas de la estructura de la base de datos



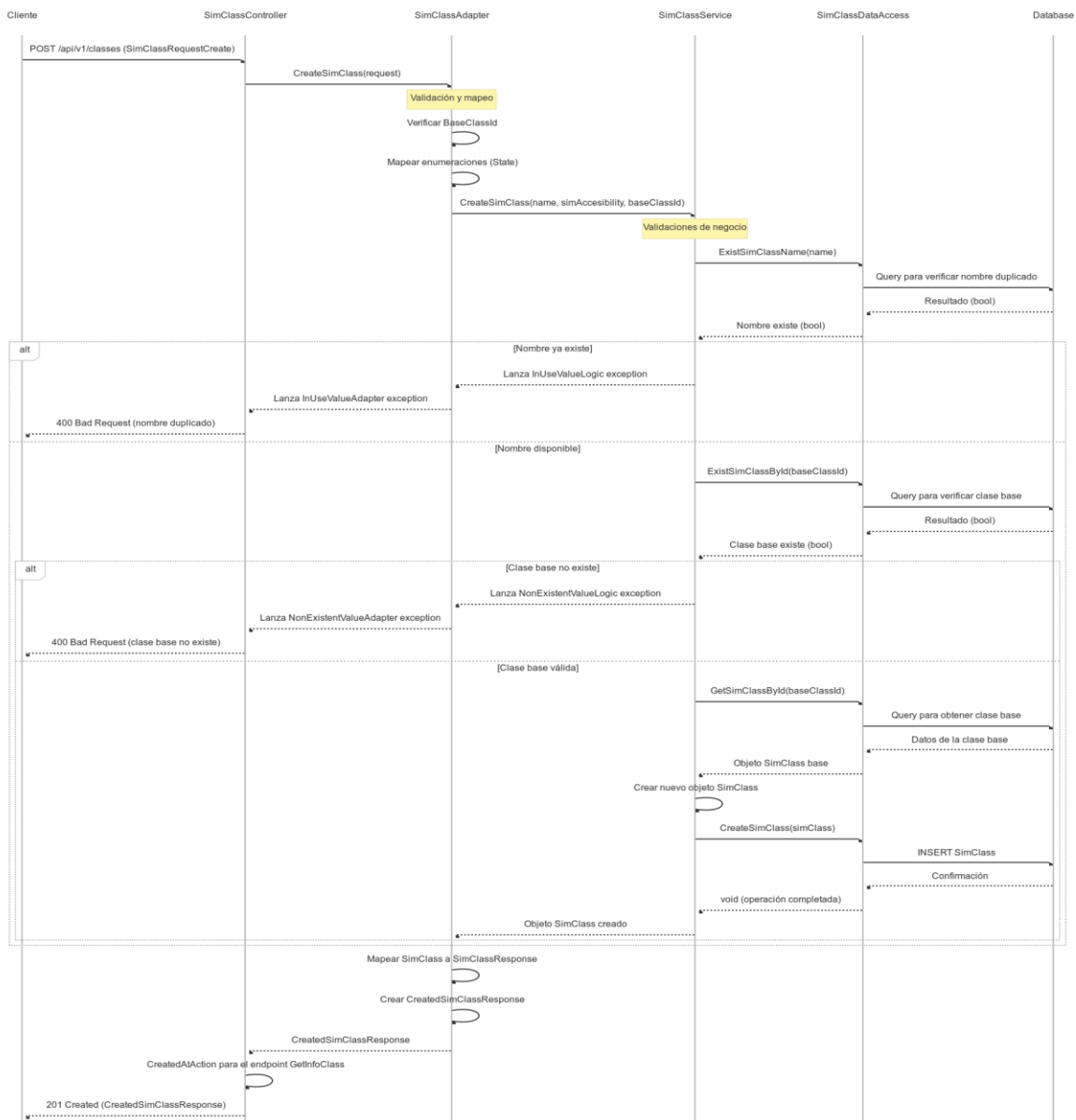
Funcionalidades de mayor relevancia

Diagramas de interacción que muestran las clases involucradas

Crear método:



Crear clase:



Justificación del diseño

Optamos por utilizar una arquitectura en capas porque ofrece una separación clara de responsabilidades dentro de la aplicación. Cada capa cumple un rol específico (ya sea presentación, lógica de negocio o acceso a datos) lo que favorece la modularidad del sistema. Esta organización permite realizar cambios en una capa sin que eso afecte directamente a las demás; lo cual es clave para mantener el sistema flexible y escalable.

Gracias a ello, también es posible reutilizar componentes en distintos contextos, lo que aporta valor en términos de eficiencia y mantenimiento. Además, al contar con una estructura bien definida, el código se vuelve más ordenado y fácil de mantener a lo largo del tiempo. Otra razón importante por la que elegimos este enfoque es la experiencia

previa que tenemos con este tipo de arquitectura, lo que nos permitió trabajar con mayor seguridad y fluidez desde el inicio del desarrollo.

Mecanismos de inyección de dependencias

En el proyecto se implementó una fábrica centralizada de servicios denominada *ServiceFactory*, donde se registran las dependencias utilizando principalmente el método *AddScoped*. Esta configuración asegura que se cree una nueva instancia de cada servicio para cada solicitud HTTP, lo que es adecuado para mantener un estado consistente a lo largo de la operación. Además, se utiliza la técnica de inyección por constructor, mediante la cual las clases reciben interfaces como parámetros, en lugar de instanciar directamente sus dependencias o depender de implementaciones concretas.

El uso de este enfoque responde a la necesidad de desacoplar las clases de sus dependencias directas, promoviendo un diseño más modular, flexible y orientado a principios de buenas prácticas como el *principio de inversión de dependencias* de SOLID. Gracias a esta estructura, se facilita el mantenimiento del código, se reducen los costos de modificación ante cambios futuros, y se simplifica la creación de pruebas unitarias, ya que las dependencias pueden ser fácilmente reemplazadas por objetos simulados o versiones alternativas.

Principios SOLID

- **Single Responsibility Principle – SRP:** Nos planteamos que cada clase debe tener una única razón para cambiar, es decir, debe tener una única responsabilidad. Para asegurarnos que no hagan cosas que “no deberían” hacer. Por cada entidad hay una clase que se encarga únicamente de sus responsabilidades necesarias, a su vez esto se divide en una clase por capa. Ejemplo: *SimClassController* (WebApi): recibe las solicitudes del usuario y solicita una respuesta para devolvérsela. *SimClassAdapter*: Traduce entre modelos de dominio y modelos de solicitud/respuesta. *SimClass* (Domain): representa la entidad *clase* del modelo de negocio con sus atributos, métodos y relaciones con otras clases. *SimClassService* (BusinessLogic): Contiene la lógica de negocio relacionada con clases. *SimClassDataAccess* (DataAccess): Maneja el acceso a datos para las clases creadas.
- **Dependency Inversion Principle – DIP:** aplicamos este principio en todo el proyecto obligatorio para lograr un diseño desacoplado. Las clases de alto nivel, como las de la capa de Business Logic, no dependen directamente de implementaciones concretas (por ejemplo, *SimClassDataAccess*), sino de interfaces (como *ISimClassDataAccess*). Esto nos permite cambiar o extender las implementaciones sin modificar la lógica de negocio, lo que hace que el sistema sea más flexible, mantenible y fácil de testear.

- **Interface Segregation Principle – ISP:** intentamos aplicar este principio, que establece que una clase no debe estar obligada a depender de interfaces que no utiliza. Sin embargo, durante el desarrollo detectamos que algunos adapters necesitaban acceder a funcionalidades distribuidas en diferentes interfaces de la capa Business Logic, lo mismo con BusinessLogic. Por razones de tiempo, no pudimos refactorizar y dividir adecuadamente esas interfaces, lo que generó ciertas dependencias innecesarias. Reconocemos que una mejor segregación de interfaces habría mejorado la cohesión y reducido el acoplamiento entre componentes.
- **Separation of Concerns – SoC:** aplicamos este principio dividiendo el sistema en capas bien definidas, cada una con una responsabilidad clara. La Web API se encarga exclusivamente de recibir y enviar datos (request/response), la capa Adapter transforma esos datos en estructuras del dominio, la Business Logic implementa las reglas del negocio, y DataAccess gestiona el acceso a la base de datos. Esta separación nos permite desarrollar, probar y mantener cada parte del sistema de forma independiente, reduciendo el acoplamiento y mejorando la claridad del código.
- **Liskov Substitution Principle – LSP:** seguimos este principio al diseñar nuestras interfaces y clases de forma que las implementaciones concretas puedan sustituir a sus abstracciones sin alterar el comportamiento esperado del sistema. Por ejemplo, los servicios y componentes que dependen de interfaces como ISimClassDataAccess o ISimClassService pueden funcionar correctamente sin importar qué implementación concreta se inyecte. Esto asegura que las clases hijas o derivadas respeten el contrato definido por sus clases base o interfaces, manteniendo la coherencia y confiabilidad del sistema.
- **Open/Closed Principle (OCP):** Usamos el principio Open/Closed en la clase Reference porque queríamos permitir agregar nuevas formas de representar desde dónde se hace una invocación de método sin tener que modificar el código existente.

En lugar de hacer una gran estructura condicional (como un if o switch) en una sola clase para manejar todas las formas posibles (this, base, atributos, parámetros, variables), creamos una clase abstracta Reference con un método como GetClass(), y luego definimos una subclase distinta para cada caso.

Esto nos permite:

Extender el comportamiento (agregando una nueva subclase como ReferenceStatic, por ejemplo)

Sin modificar el código de las subclases anteriores ni de Reference.

Usar polimorfismo para que el comportamiento correcto (por ejemplo, cómo se resuelve GetClass()) se elija automáticamente según el tipo de referencia.

Otros principios de diseño utilizados

- **Principio de Encapsulación:** Como ya mencionamos anteriormente, procuramos los detalles internos de cada clase estén ocultos, exponiendo solo lo necesario. Esto lo hicimos con el fin de proteger la integridad de los datos y reduce el riesgo de errores.
- **Repositorio:** utilizamos este patrón para separar la lógica de acceso a la base de datos del resto del sistema, promoviendo la cohesión y el desacoplamiento. Definimos una interfaz `IDataAccess` que expone operaciones CRUD (crear, leer, actualizar y borrar), y su implementación concreta `DataAccess`, que interactúa directamente con la base de datos. Además de las operaciones básicas, incluimos consultas más complejas pensamos que mejoraríamos el rendimiento de nuestra aplicación con ello.
- **Fábrica:** Aplicamos este patrón en `ServiceFactory` esta clase actúa como una fábrica de configuración de servicios, centralizando el registro de todas las dependencias necesarias para la aplicación. A través del método `AddServices`, se registran en el contenedor de dependencias las implementaciones de los adapters, servicios de negocio y acceso a datos, cada uno con su respectiva interfaz. Esto permite que los controladores de la Web API reciban las dependencias necesarias sin conocer sus implementaciones concretas, promoviendo el desacoplamiento y la inyección de dependencias. Además, se configura el contexto de base de datos `SimulatorDbContext` usando `Entity Framework Core` con una conexión `SQL Server`.
- **Adapter:** usamos este patrón para desacoplar la capa de `BusinessLogic` de los modelos específicos expuestos por la Web API. En lugar de que la lógica de negocio conozca o dependa directamente de los modelos que usa la API (por ejemplo, DTOs en nuestro caso le llamamos `Models (Request,Response)`), implementamos adapters que transforman esos modelos en estructuras internas que la lógica de negocio puede manejar (clases de `Domain`). De esta forma, la Web API puede recibir o enviar datos con sus propios modelos, mientras que internamente se usan objetos adaptados para procesar la información. Esto mejora el aislamiento entre capas y facilita cambios o validaciones específicas en cada una.

Decisiones propias de diseño

1. Cuando un método tiene una invocación o es sobrescrito por otra clase, ya no es posible agregarle nuevos parámetros.
2. Lo que identifica a un método como único es su nombre junto con el tipo de sus parámetros, respetando el orden en el que se insertaron, al igual que en los lenguajes de programación.

3. Es posible ejecutar un método a través de un parámetro, atributo o variable local, incluso si el tipo de clase de estos es de clase abstracta. Esta decisión se basa en la consigna de los ejemplos proporcionados por la cátedra.
4. Al momento de crear una clase, únicamente se debe ingresar el ID de su clase base, su estado y su nombre. Los métodos y atributos se agregan posteriormente a través de *endpoints* específicos. Al modificar una clase, es posible actualizar tanto los métodos y atributos como los datos propios de la clase en un mismo JSON.
5. En los casos en que se hereda de una clase abstracta que contiene métodos abstractos, el sistema devolverá un error si no se implementan dichos métodos. Por eso, para poder definir correctamente una clase concreta (ya sea normal o sealed), se recomienda primero crearla con una clase base que no sea abstracta, y luego modificarla para establecer la clase base abstracta e implementar los métodos correspondientes.
6. Relacionados con lo anterior, para la creación de cada elemento (clases, métodos o atributos) se realiza de forma individual y siempre asociada a su correspondiente "dueño", con el objetivo de facilitar el manejo de los datos mediante estructuras JSON.
7. No se permite eliminar clases, métodos o atributos que estén siendo utilizados por otras entidades del sistema.
8. Para la ejecución de métodos se optó por implementar un *endpoint* con el verbo *HTTP POST*. Esta elección se basa en la necesidad de simplificar la consulta (dado que requiere múltiples parámetros) así como en la intención de registrar cada ejecución en la base de datos (*log*) y así garantizar la propiedad de idempotencia de la operación.
9. Se implementó un Action Filter para validación personalizada de los datos de entrada en el JSON de nuestra web api, para permitir mostrar mensajes de campos requeridos personalizados para que el usuario sepa que falta.

Descripción del mecanismo de acceso a datos

Para mapear entidades utilizamos Eager Loading, en DataAccess realizamos consultas complejas, en las que se requería por ejemplo de un SimMethod ordenar por índice sus parámetros, y también sus invocaciones, e igualarlo a otro método para ver si son iguales. Además, tuvimos que realizar consultas recursivas para determinar si un método es heredado navegando a través de sus clases base. También para determinar si un atributo es heredado por una clase, o que clase hereda de cierta clase.

Para realizar lo anterior teníamos que cargar en memoria todas las propiedades de dichas clases, ya que estas consultas involucraban objetos dentro de otros objetos.

Debido a que es importante el orden de parámetros y invocaciones de un método, guardamos en un atributo de las clases que representan esto un atributo Index el cual

representa el índice dentro de la lista de la entidad que lo contiene. Esto lo hicimos porque en memoria se guardan bien los órdenes de las listas, pero al ponerlas en una base de datos y mapearlos, el orm entity framework los mapea por orden de id por defecto.

Para el mapeo de herencia de Referencia utilizamos TPH (Table Per Hierarchy) con el objetivo de minimizar la cantidad de tablas en la base de datos, lo que simplifica el esquema y mejora el rendimiento al evitar joins innecesarios al consultar subtipos. Elegimos esta estrategia porque, en nuestro caso, la presencia de columnas nulas no representa un problema, ya que las subclases comparten la mayoría de los atributos.

Para interactuar con la base de datos utilizamos la clase *DbContext*, a través de repositorios que encapsulan las operaciones comunes como agregar, eliminar, consultar y modificar datos. Estos repositorios siguen una estructura basada en interfaces, lo que nos permite definir un contrato para cada uno y luego implementar sus versiones concretas de forma flexible.

Decidimos crear un repositorio “DataAccess” por funcionalidad principal en nuestro proyecto. Cada repositorio es inyectado en el servicio correspondiente, promoviendo una arquitectura desacoplada y fácil de mantener. Este enfoque no solo mejora la claridad del sistema, sino que también favorece la reutilización y testeo de los componentes de acceso a datos.

Descripción del manejo de excepciones

Decidimos utilizar excepciones personalizadas para representar errores específicos del dominio del sistema simulador. Esto permite una mayor claridad en los errores retornados.

Cada excepción personalizada se mapea con un código HTTP y un código de error interno (InnerCode) que ayuda a identificar el tipo exacto de error del lado del cliente o de sistemas consumidores.

Estas excepciones son capturadas mediante un filtro global de excepciones, lo cual permite mantener los controladores limpios, sin necesidad de capturar errores manualmente mediante try-catch.

En el exception filter de la web api solo se capturan las excepciones de “Adapter”, en la clase Adapter se transforman las excepciones de Domain y de BussinesLogic a su correspondiente equivalente en Adapter, esto lo hicimos para que la web api solo dependa de Adapter y desacoplarla del resto de los paquetes.

Excepciones utilizadas en el filter:

- **InvalidAttributeAdapter**

Se lanza cuando un atributo recibido en una operación no cumple con las reglas

del sistema.

Retorna 400 Bad Request, InnerCode = 1

- **NonExistentValueAdapter**

Se lanza cuando se intenta operar sobre una entidad o valor que no existe en el sistema.

Retorna 404 Not Found, InnerCode = 2

- **InvalidExecutionAdapter**

Se lanza cuando se intenta ejecutar una acción inválida en el contexto actual del sistema.

Retorna 400 Bad Request, InnerCode = 3

- **InUseValueAdapter**

Se lanza cuando se intenta eliminar o modificar un valor que se encuentra en uso por otra entidad.

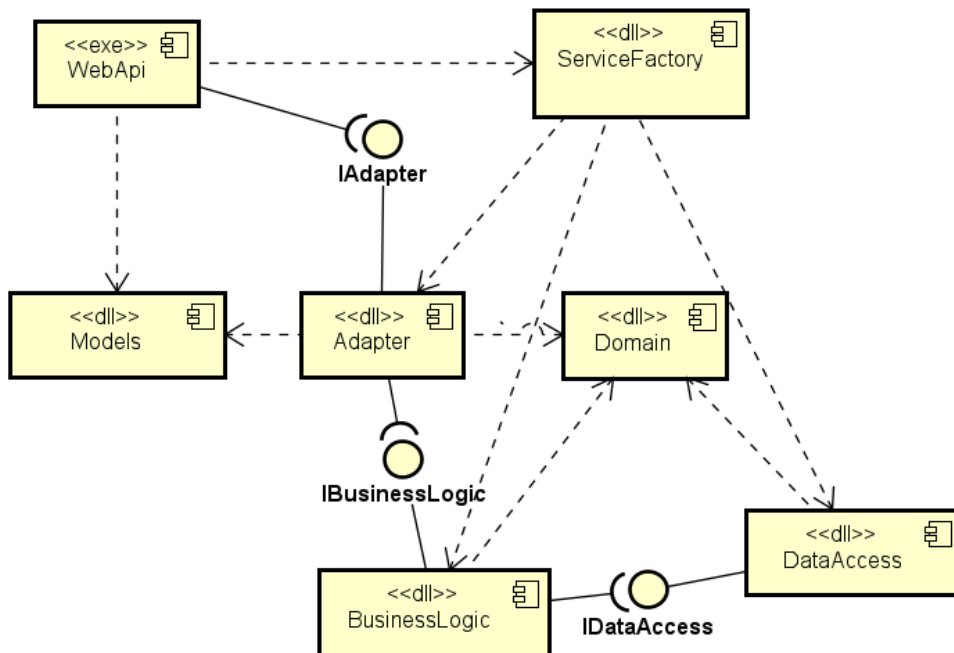
Retorna 409 Conflict, InnerCode = 4

- **Exception** (por defecto)

Cubre cualquier otra excepción no especificada anteriormente.

Retorna 500 Internal Server Error, InnerCode = 5

Diagrama de componentes



Arquitectura desacoplada: la API usa IAdapter para comunicarse con los adaptadores, que transforman las solicitudes y llaman a la lógica de negocio (IBusinessLogic); esta a su vez delega la persistencia en IDataAccess. Todas las dependencias apuntan a interfaces, cumpliendo los principios de Dependency Inversion y Separation of Concerns.